

RÚBRICA DE EVALUACIÓN DEL PRODUCTO 1 - UNIDAD 2

DESARROLLO DE APLICACIONES WEB II (5° ciclo)

Integrantes del grupo	

Fecha:

CRITERIO/NIVEL	NULO (0)	INICIO(1)	EN PROCESO (2)	LOGRADO (3)	AVANZADO (4)	PUNTAJE OBTENIDO
Definición del Estilo Arquitectónico	No define ninguna arquitectura o estructura para el proyecto.	Menciona un nombre de arquitectura (ej. MVC) pero el código o diagramas no corresponden a dicho estilo.	Define un estilo arquitectónico, pero la separación de responsabilidades es confusa en varios puntos del diseño.	Define con claridad el estilo arquitectónico elegido (MVC, Capas, etc.), respetando sus reglas básicas de flujo.	Arquitectura de alto nivel. Define y documenta un estilo arquitectónico coherente, modular, robusto con manejo de rutas amigables y seguridad perimetral.	
Fundamentación y Justificación Técnica	No justifica por qué eligió dicha estructura.	La justificación se limita a "porque así se enseñó en clase" o conceptos vagos sin sustento técnico.	Justifica la arquitectura mencionando beneficios genéricos, pero no explica por qué es la mejor opción para su proyecto específico.	Justifica la elección arquitectónica analizando las necesidades del proyecto (mantenibilidad, rapidez, seguridad).	Justificación Crítica. Argumenta la elección comparando el estilo elegido frente a otros, demostrando dominio de pros y contras técnicos.	
Arquitectura de la Capa de Datos	No presenta un diseño para la gestión de datos.	El diseño de datos es monolítico y se mezcla con la interfaz de usuario o la lógica de control.	Propone una estructura para los datos, pero no hay una abstracción clara (ej. el SQL sigue muy acoplado a la lógica).	Diseña una lógica de persistencia coherente con la arquitectura elegida e implementa un mini ORM.	Capa de persistencia avanzada: implementa un Query Builder encadenable y métodos reutilizables como all(), find() y where()	
Organización del Código y Estándares de Estructura	No hay un orden lógico en los archivos del proyecto. No usa namespaces ni autoloading.	Presenta una organización de carpetas básica pero inconsistente; los nombres de archivos no siguen un estándar (ej.: PSR-4)	Organiza los archivos en directorios y logra implementar Autoloading, pero tiene conflictos de colisión o no sigue un estándar claro.	Organización profesional con Namespaces jerárquicos y carga automática funcional mediante spl_autoload_register	Estructura Óptima. Código limpio y desacoplado siguiendo estándares tipo PSR-4. Organización impecable que facilita la navegación.	
Verificación de Funcionalidad	No demuestra funcionalidad ni pruebas realizadas.	Los endpoints existen pero retornan errores de servidor (500) o no procesan el JSON.	Procesa peticiones pero no valida datos de entrada ni maneja códigos de estado HTTP adecuados.	Los endpoints son funcionales y devuelven datos en formato JSON verificables mediante Postman.	Pruebas exitosas de CRUD completo en Postman; uso correcto de verbos HTTP y validación de URLs amigables.	
TOTAL:						0